

SKILL WEEK 12

Task 1:

Code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\nSignal %d received. Handler executed.\n", sig);
}

int main() {
    signal(SIGUSR1, handler);
    printf("PID: %d\n", getpid());
    printf("Waiting for signal...\n");
    sleep(8);
    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 12.1.c
venkat038@AYYAPPA:~$ gcc 12.1.c -o 12.1
venkat038@AYYAPPA:~$ ./12.1
PID: 4401
Waiting for signal...

Signal 30 received. Handler executed.
```

SKILL WEEK 12

Task 2:

Code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig) {
    printf("\nSIGINT received. Foreground job terminated safely.\n");
}

int main() {
    signal(SIGINT, handle_sigint);
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C...\n");

    for (int i = 1; i <= 5; i++) {
        printf("Running: %d\n", i);
        sleep(1);
    }

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 12.2.c
venkat038@AYYAPPA:~$ gcc 12.2.c -o 12.2
venkat038@AYYAPPA:~$ ./12.2
PID: 4405
Press Ctrl+C...
Running: 1
Running: 2
^C
SIGINT received. Foreground job terminated safely.
```

SKILL WEEK 12

Task 3:

Code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigtstp(int sig) {
    printf("\nSIGTSTP received. Job state preserved.\n");
}

int main() {
    signal(SIGTSTP, handle_sigtstp);
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z...\n");

    for (int i = 1; i <= 5; i++) {
        printf("Running: %d\n", i);
        sleep(1);
    }

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 12.3.c
venkat038@AYYAPPA:~$ gcc 12.3.c -o 12.3
venkat038@AYYAPPA:~$ ./12.3
PID: 4410
Press Ctrl+Z...
Running: 1
Running: 2
^Z
SIGTSTP received. Job state preserved.
```

SKILL WEEK 12

Task 4:

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        setpgid(0, 0);
        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", getpgid());
        sleep(2);
        return 0;
    }

    wait(NULL);
    printf("Parent PID: %d\n", getpid());
    printf("Child process group completed.\n");
    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 12.4.c
venkat038@AYYAPPA:~$ gcc 12.4.c -o 12.4
venkat038@AYYAPPA:~$ ./12.4
Child PID: 4415
Process Group ID: 4415
Parent PID: 4414
Child process group completed.
```